

整合練習 3：BAPI 的 COMMIT / ROLLBACK 與 LUW 控制（延續 QA12 案例）

Lecture

if02 讀 `BAPI_INSPLLOT_SETUSAGEDECISION` 原始碼時看到一段註解，指名「成功要呼叫 `BAPI_TRANSACTION_COMMIT`，失敗要呼叫 `BAPI_TRANSACTION_ROLLBACK`」。這題把這兩支 FM 的原始碼也讀出來，你會發現一件很多人沒想過的事：它們一點都不神秘。

```
FUNCTION BAPI_TRANSACTION_COMMIT
IMPORTING
    VALUE(WAIT) LIKE BAPITA-WAIT OPTIONAL
EXPORTING
    VALUE(RETURN) LIKE BAPIRET2.

IF WAIT EQ SPACE.
    COMMIT WORK.
ELSE.
    COMMIT WORK AND WAIT.
IF SY-SUBRC NE 0.
    CALL FUNCTION 'BALW_BAPIRETURN_GET2'
        EXPORTING
            TYPE      = 'E'
            CL        = 'S&'
            NUMBER    = '150'
        IMPORTING
            RETURN    = RETURN.
ENDIF.
ENDIF.
CALL FUNCTION 'BUFFER_REFRESH_ALL'.
ENDFUNCTION.
```

```
FUNCTION BAPI_TRANSACTION_ROLLBACK
EXPORTING
    VALUE(RETURN) LIKE BAPIRET2.

ROLLBACK WORK.
CALL FUNCTION 'BUFFER_REFRESH_ALL'.
ENDFUNCTION.
```

`BAPI_TRANSACTION_COMMIT` 就是 `COMMIT WORK`（或 `COMMIT WORK AND WAIT`）包一層 `RETURN` 結構；
`BAPI_TRANSACTION_ROLLBACK` 就是 `ROLLBACK WORK`——基礎課 ex23 已經教過的兩句最原始的 ABAP LUW 語句，這裡沒有任何額外魔法。之所以要包成一支 BAPI，是因為 BAPI 的呼叫端不一定是 ABAP 程式（可能是外

部系統透過 RFC 呼叫)，外部呼叫端沒辦法直接下 **COMMIT WORK** 這種 ABAP 關鍵字，只能呼叫一支「效果等同於 **COMMIT WORK**」的 FM。這就是本題最重要的觀念：BAPI 的交易控制不是另一套獨立機制，是把 ex23 教過的東西包裝成「外部系統也能觸發」的形式。

WAIT 參數的意義，直接對照 if02 挖到的 **QAMB_COLLECT_RECORD** 延遲寫入機制就很好懂：

- **WAIT = SPACE** (不傳或傳空) → **COMMIT WORK**：非同步，這句執行完，資料庫更新請求已經送出去，但不保證「已經真的寫完」——ABAP 的 Update Task (V1/V2) 可能還在背景跑
- **WAIT = 'X'** → **COMMIT WORK AND WAIT**：同步，這句會等到 Update Task 真正執行完 (成功或失敗) 才返回，**sy-subrc** 才能可靠反映「有沒有 Update 失敗」

if02 讀到 **QAMB_COLLECT_RECORD** 是靠 **PERFORM SICHERN_QAMB ON COMMIT** 觸發、最終 **CALL FUNCTION 'QEVA_MATERIALDOC_TO_LOT' IN UPDATE TASK** 才真正寫進 **QAMB** 表——這正是「非同步 Update」的實際案例。如果情境 B 補過帳的程式呼叫完這四支 FM 後，用 **WAIT = SPACE** 就 **COMMIT**，接著馬上又下一句 **SELECT FROM qamb** 想確認剛剛的關聯有沒有寫進去，**有機率讀到舊資料** (Update Task 可能還沒跑完)；要保證讀得到，**COMMIT** 這一步就要用 **WAIT = 'X'**。

BUFFER_REFRESH_ALL 這一行也值得注意：**COMMIT / ROLLBACK** 之後都呼叫了它，因為 SAP 應用伺服器對很多表 (尤其是 Customizing / 主檔類的表) 會做 Buffer (快取在應用伺服器記憶體，減少每次都問資料庫)，如果剛剛的異動可能影響到被緩衝的表，不刷新緩衝區，同一個使用者 session 接下來的查詢可能還是看到異動前的舊資料——這是另一層「呼叫完不代表馬上到處都看得到最新結果」的細節，跟 **WAIT** 參數處理的是不同層面的問題 (**WAIT** 管的是資料庫本身的 Update Task 有沒有跑完，**BUFFER_REFRESH_ALL** 管的是應用伺服器記憶體裡的快取)。

LUW 邊界設計，回到 if02 的兩個情境：

- 情境 A：一支 **BAPI_INSPLLOT_SETUSAGEDECISION** 呼叫完，緊接著一次 **BAPI_TRANSACTION_COMMIT** (失敗就 **BAPI_TRANSACTION_ROLLBACK**) ——LUW 邊界很單純，一次呼叫、一次決定
- 情境 B：**MB_CREATE_GOODS_MOVEMENT** → **MB_POST_GOODS_MOVEMENT** → **QAMB_COLLECT_RECORD** → **STATUS_CHANGE_INTERN** 這四支必須全部包在同一個 **LUW**，最後才一次 **COMMIT WORK**——這是典型的「多步驟、要嘛全部生效要嘛全部不生效」情境 (呼應基礎課 ex23 教過的 **ROLLBACK WORK all-or-nothing**)：只要 **STATUS_CHANGE_INTERN** 這一步因為某個狀態衝突失敗，前面已經「呼叫成功」的過帳跟 **QAMB** 登記如果沒有一起 **ROLLBACK**，就會出現「物料憑證真的過帳了、但 **QAMB** 沒登記、檢驗批狀態也沒更新」這種資料不一致的半套結果——比 ex23 訂單 Header/Detail 兩張表的情境更複雜，因為這裡有四個不同 Function Group 的呼叫，任何一步都可能是失敗點

學習目標

- 能解釋 **BAPI_TRANSACTION_COMMIT / BAPI_TRANSACTION_ROLLBACK** 底層就是 **COMMIT WORK [AND WAIT] / ROLLBACK WORK**，不是獨立於 ABAP LUW 之外的另一套機制
- 能講出 **WAIT** 參數控制的是「要不要等 Update Task 真正跑完」，並舉出「不等待可能讀到舊資料」的具體場景
- 能設計出情境 B (四支 FM 序列) 正確的 LUW 邊界：所有步驟成功才 **COMMIT**，任何一步失敗就整串 **ROLLBACK**
- 理解 **BUFFER_REFRESH_ALL** 處理的是應用伺服器緩衝，跟 **WAIT** 處理的資料庫 Update Task 是兩個不同層面的「資料還沒到位」問題

事前準備

不需要新建任何 SAP 物件，延續 if02 已查證的物件，另外新增查證兩支：

FM	Function Group	套件
BAPI_TRANSACTION_COMMIT	BAPT	SBAPI
BAPI_TRANSACTION_ROLLBACK	BAPT	SBAPI

(if02 已查證的 BAPI_INSPLT_SETUSAGEDECISION / MB_CREATE_GOODS_MOVEMENT / MB_POST_GOODS_MOVEMENT / QAMB_COLLECT_RECORD / STATUS_CHANGE_INTERN / BAPI_GOODSMVT_CREATE / BAPI_GOODSMVT_CANCEL 這題繼續沿用)

題目需求

1. 對照原始碼：指出 BAPI_TRANSACTION_COMMIT 的 IF WAIT EQ SPACE 分支對應基礎課 ex23 教過的哪一句 ABAP 語句，ELSE 分支又對應哪一句。
2. WAIT 情境分析：延續 if02 讀到的 QAMB_COLLECT_RECORD → PERFORM ... ON COMMIT → IN UPDATE TASK 這條延遲寫入鏈，說明「情境 B 補過帳程式呼叫完 COMMIT 之後馬上查 QAMB 表」這件事，WAIT = SPACE 跟 WAIT = 'X' 分別可能發生什麼結果。
3. 完成 if02 的類別骨架設計：把 if02 思考題 5 設計的 ZCL_IF02_INSPLT_POSTING 方法骨架，在正確的位置補上 BAPI_TRANSACTION_COMMIT / BAPI_TRANSACTION_ROLLBACK (或情境 B 用 COMMIT WORK / ROLLBACK WORK，兩種寫法都要能講出為什麼這裡兩者等價) 的呼叫時機——只寫呼叫順序的骨架 / 註解，不用寫完整實作。
4. 失敗情境設計：情境 B 的四支 FM 序列中，如果 STATUS_CHANGE_INTERN 回傳 STATUS_NOT_ALLOWED (例如檢驗批目前的狀態不允許再標記成已過帳)，程式該怎麼處理，才不會留下「過帳成功但狀態沒更新」的半套結果？

團隊實務備註

- BAPI_TRANSACTION_COMMIT / BAPI_TRANSACTION_ROLLBACK 的原始碼比想像中短很多 (各自不到 20 行)，這對教學是件好事——直接讀原始碼比看任何教材說明都更能破除「BAPI 交易控制是什麼特殊機制」的迷思，這題刻意把完整原始碼貼出來就是為了這個效果。
- BAPI_TRANSACTION_COMMIT 的錯誤訊息用的是 BALW_BAPIRETURN_GET2 (TYPE='E' CL='S&' NUMBER='150')，跟 if02 BAPI_INSPLT_SETUSAGEDECISION 用的 BALW_BAPIRETURN_GET1 是同一個家族的工具 FM (把 sy-msgty/sy-msgid/sy-msgno 這類系統訊息欄位轉換成 BAPIRET2 / BAPIRETURN1 結構)，出題時若要示範「怎麼把一般訊息包成 BAPI RETURN 格式」，BALW_BAPIRETURN_GET1/GET2 這兩支是現成的標準工具，值得另外介紹。
- COMMIT WORK AND WAIT 讓 sy-subrc 能可靠反映 Update Task 成敗，這個特性代價是效能——同步等待 Update Task 跑完，比純 COMMIT WORK (送出去就繼續往下跑) 慢；情境 A/B 這種「呼叫端需要確認資料真的落地才能往下做事」的場景值得用 WAIT，但如果是高頻率、不需要立即確認的批次寫入，一路都用 WAIT 可能造成不必要的效能損耗，這是實務上的取捨 (if04 的 Data Conversion 批次情境會再碰到類似的效能考量)。

思考題

1. 如果情境 A (BAPI_INSPLT_SETUSAGEDECISION) 呼叫完，return-type 是 E (失敗)，但程式忘記呼叫 BAPI_TRANSACTION_ROLLBACK、直接讓程式結束，會發生什麼事？ (提示：原始碼裡提到失敗時「to e.g. dequeue the locked data」——沒有明確 ROLLBACK 或 COMMIT，這個 LUW 遺留的鎖定 / 未決

異動狀態要等到什麼時候才會被清掉？可以對照基礎課學過的 SAP LUW 生命週期，思考如果呼叫端是一支長時間跑的批次程式，忘記處理會有什麼後果）

2. `MB_CREATE_GOODS_MOVEMENT` / `MB_POST_GOODS_MOVEMENT` 這兩支透過 Function Group 全域變數溝通（if02 學到的），如果中間插了一個 `COMMIT WORK`（例如某段防呆邏輯誤觸發了一次不該有的 `COMMIT`），對這兩支 FM 之間共享的全域資料會有什麼風險？（提示：`COMMIT WORK` 除了觸發資料庫層級的異動確認，也可能牽動 SAP LUW 生命週期裡跟這次交易相關的其他資源；這題不要求查證確切機制，重點是練習「LUW 邊界不能隨便被中途打斷」這個警覺）
3. `BUFFER_REFRESH_ALL` 沒有帶任何參數，代表它會刷新「所有」緩衝的表，而不是只刷新這次異動用到的那幾張。如果一支程式在一個緊湊迴圈裡，每過帳一筆就呼叫一次 `BAPI_TRANSACTION_COMMIT`，這個設計可能有什麼效能隱憂？（提示：這正好呼應 if04 要教的「批次處理」設計取舍——逐筆 `COMMIT` vs 累積到一定量再一次 `COMMIT`）

答案

不新建任何 SAP 物件；`BAPI_TRANSACTION_COMMIT` / `BAPI_TRANSACTION_ROLLBACK` 完整原始碼已直接引用於本題 Lecture（`sap-adt` MCP 於 client 130 查證，Function Group `BAPT`，套件 `SBAPI`），未經改寫。本題延續 if02 的物件與設計題，同樣只做設計分析，未對任何真實檢驗批或物料憑證執行過帳測試。